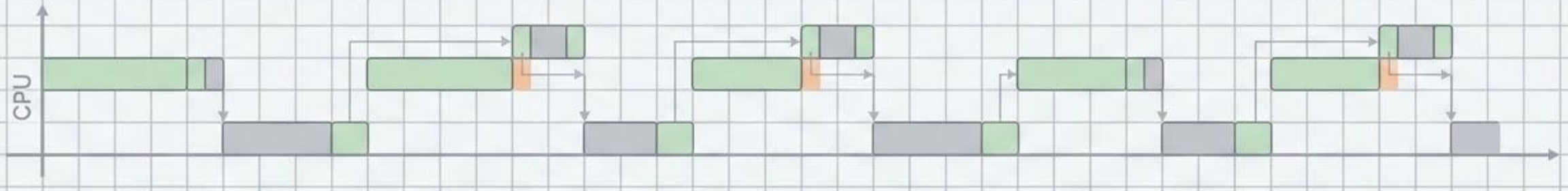
Context-Aware Deadline Scheduling Under Memory Contention

Final Project Progress & Implementation Review



Context:	CMP720 Embedded System Design
Track:	Optimization

Team:	Umut Kalman & Eren Karadağ

Problem Definition and Baseline Recap

Rate Monotonic (RM)	
✅	Static priority based on task frequency
✅	Simple RTOS implementation
✅	Lower maximum CPU utilization boundary

→

Earliest Deadline First (EDF)	
✅	Dynamic priority based on closest absolute deadline
✅	Theoretically allows up to 100% uniprocessor utilization
✅	Assumes perfect hardware isolation

The Hidden Flaw: Perfect hardware isolation is a myth.

The Baseline: EDF offers 100% theoretical utilization via dynamic absolute deadline prioritization.

The Problem: Standard EDF logic is blind to microarchitectural realities like cache thrashing.

The Result: Proven schedulable systems suffer jitter and sporadic deadline misses.

Our Approach: A user-space FreeRTOS wrapper combining EDF safety with contention awareness.

Mathematical Taxonomy and Decision Variables

Task Properties	
T_i	Task Period (Time interval between consecutive job arrivals)
D_i	Relative Deadline (Time by which a job must finish after arrival)
C_i	Worst-Case Execution Time (WCET) in strict isolation
M_i	Memory Intensity factor of task i (Profiling-derived constant)
System States	
d_b	Absolute Deadline of job b (Arrival Time + D_b)
f_b	Actual forecasted completion time of job b
$C_{rem,j}(t)$	Remaining execution time for job j at current time t
Algorithmic Parameters	
$Slack_j(t)$	Dynamic safe delay time available for job j at time t

Core Scheduling Equations

The Golden Rule (Slack)

Constraint 1 (Compliance): Completion time must never exceed absolute deadline.

$$f_b \leq d_b$$

Constraint 2 (Slack Proxy): Dynamic safe delay time available.

$$Slack_j(t) = d_j - t - C_{rem,j}(t)$$

The Penalty Function

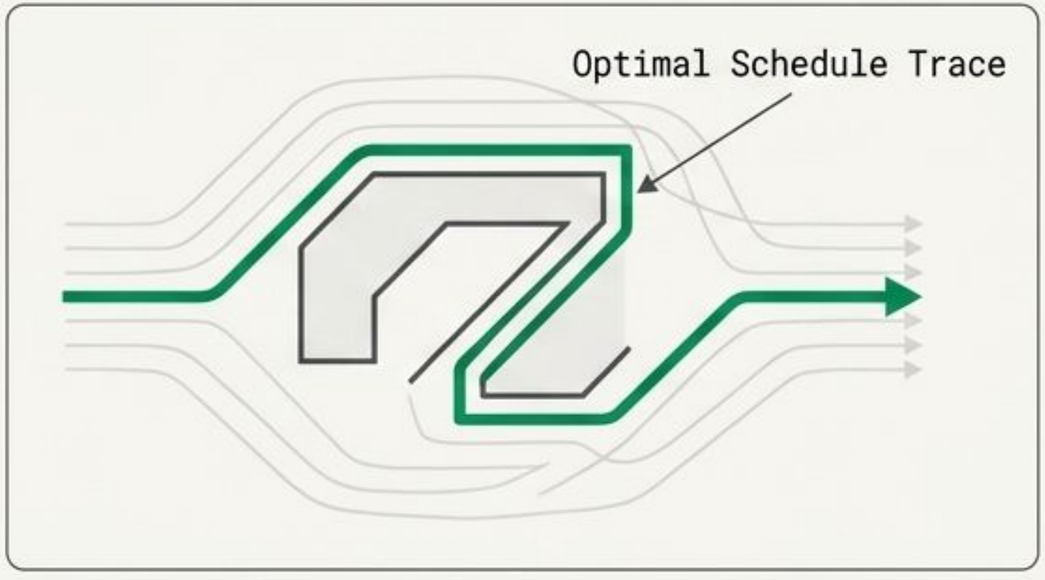
$$P(j_a, j_b) = \alpha M_a M_b$$

Objective: Minimize pairwise cumulative memory penalty across consecutively running tasks.

The Golden Rule: **High-memory tasks are ONLY deferred if Slack > 0.**

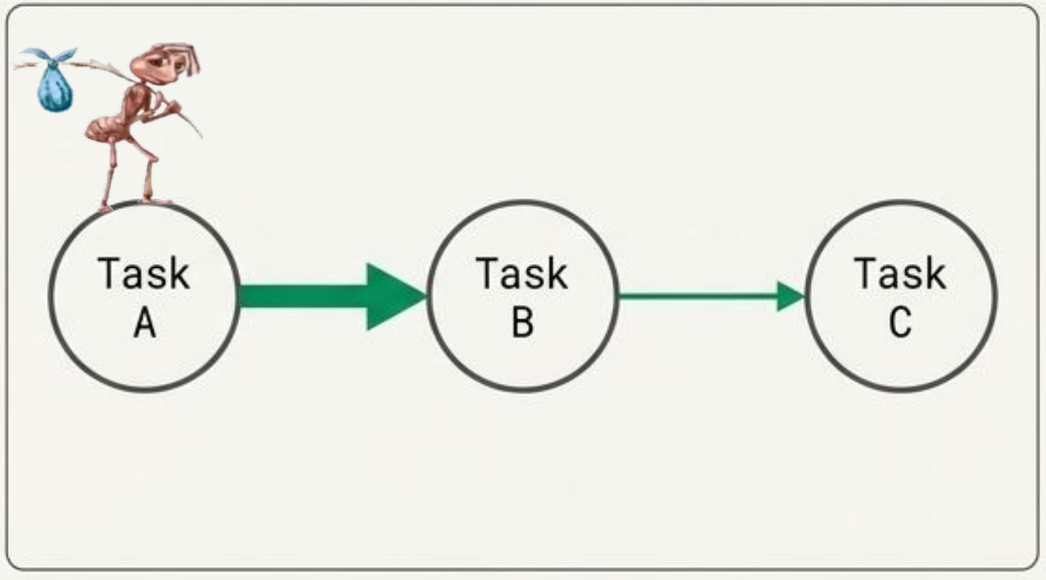
Benchmarking Validation via Ant Colony Optimization

The Metaheuristic Concept



Ant Colony Optimization (ACO) relies on "**ants**" constructing candidate schedules (dispatch traces). High-quality schedules reinforce **pheromones** on specific job-to-job or segment-to-job transitions.

The Sequencing Application



Memory contention is fundamentally a sequencing problem (who runs after whom).

ACO excels by rewarding optimal pairings over millions of iterations.

Rigorous Disclaimer: ACO is deployed strictly as an **offline near-optimal** reference solver to benchmark the **online wrapper**. It is not claimed to be absolute mathematical optimal.

Offline ACO Formulation & Cost Dynamics

Variables:

- τ_{ab}: pheromone value a→b
- η_{ab}: heuristic desirability
- A: set of candidate ready jobs
- p_{ab}: probability a→b
- ρ: evaporation rate
- Q: deposit weight
- $Cost(S)$: schedule cost
- $\Delta\tau_{ab}$: pheromone deposit

Selection Probability

$$p_{ab} = \frac{\tau_{ab}^{\alpha_{aco}} \cdot \eta_{ab}^{\beta_{aco}}}{\sum_{k \in A}(\tau_{ak}^{\alpha_{aco}} \cdot \eta_{ak}^{\beta_{aco}})}$$

Balances historical trail strength against heuristic desirability.

Evaporation

$$\tau_{ab} \leftarrow (1-\rho)\tau_{ab}$$

Prevents premature convergence on sub-optimal sequences.

Deposit

$$\Delta\tau_{ab} = \frac{Q}{Cost(S)}$$

Applied as $\tau_{ab} \leftarrow \tau_{ab} + \Delta\tau_{ab}$

Rewards transitions within successful dispatch traces.

Cost Function

$$Cost(S) = w_{\text{miss}}(\text{deadline\_misses}) + w_{\text{late}}(\text{total\_lateness}) + w_{\text{cont}}(\text{contention\_penalty}) + w_{\text{smooth}}(\text{smooth\_memory\_penalty})$$

(Optional response/segment costs for preemptive mode).

Cost Weighting Priority: w_{miss} heavily dominates the cost function. Memory penalties (w_{cont}, w_{smooth}) act as secondary, fine-tuning optimization goals.

Discussion: The Midterm Pivot

The 3-Task Midterm Setup

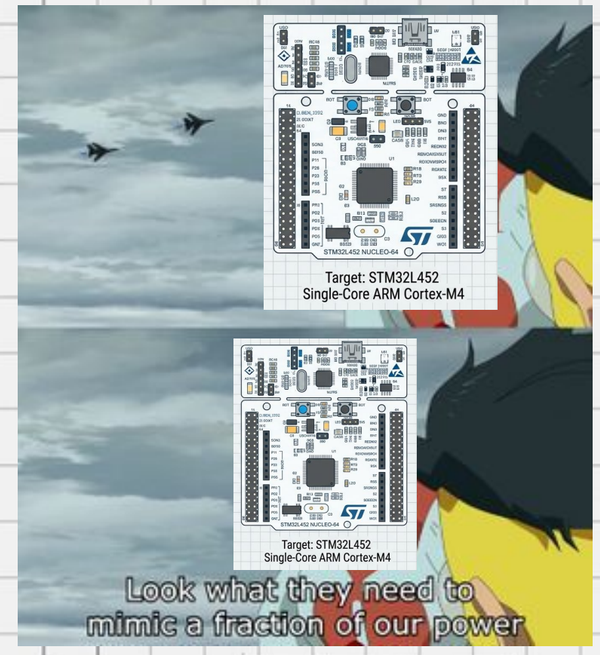
Diagnostic Breakdown: The Under-Utilization Problem	
The Midterm Reality:	The 3-task workload was inadequate for this research setup.
The Cause:	Abundant natural slack. The probability of the high-memory Crypto task preempting a sensitive Control task was mathematically negligible.
The Result:	The context-aware scheduler had nothing to optimize. Pure EDF handled the workload perfectly without ever triggering the predictive penalty wrapper.

Hardware Deployment and Software Ecosystem

Application Layer (Custom .cpp)

Motor Control (Low Mem)	Sensor ADC (Low Mem)	Crypto AES (High Mem)	Vision Processing (High Mem)

Custom wrappers utilizing Object-Oriented Programming (OOP) for clean, polymorphic scheduler swapping.



Proposed Scheduler Wrapper (User-Space)

Modern **CMake**-based build system natively integrated with the **FreeRTOS** Kernel (Preemptive Tick Scheduling).

Hardware Deployment

STM32L452 Nucleo-64 (Single-Core ARM Cortex-M4)

Auto-generated by STM32CubeMX for baseline peripheral configuration.

Execution Interception and Workload Initialization

Boot Interception: Custom logic is injected after hardware init but strictly before the kernel takes control.

1. **Hardware Init** (Peripheral Setup)
2. **Custom C++ Entry** (Interceptor Node)
3. **ARM DWT Counter Start** (Nanosecond Profiling)
4. **osKernelInitialize()** (FreeRTOS takes over)

Mixed-Criticality Workload Models

- **Motor Control** — High Priority, Low Mem
- **Sensor Acquisition** — Mid Priority, Low Mem
- **Crypto Encryption** — Low Priority, High Mem
- **Vision Processing** — High-Mid Priority, High Mem

Deterministic Execution and Telemetry Output

Deterministic Workloads: Tasks utilize calibrated busy-spin loops to simulate heavy, highly predictable execution times, isolating scheduler behavior from random code branches.

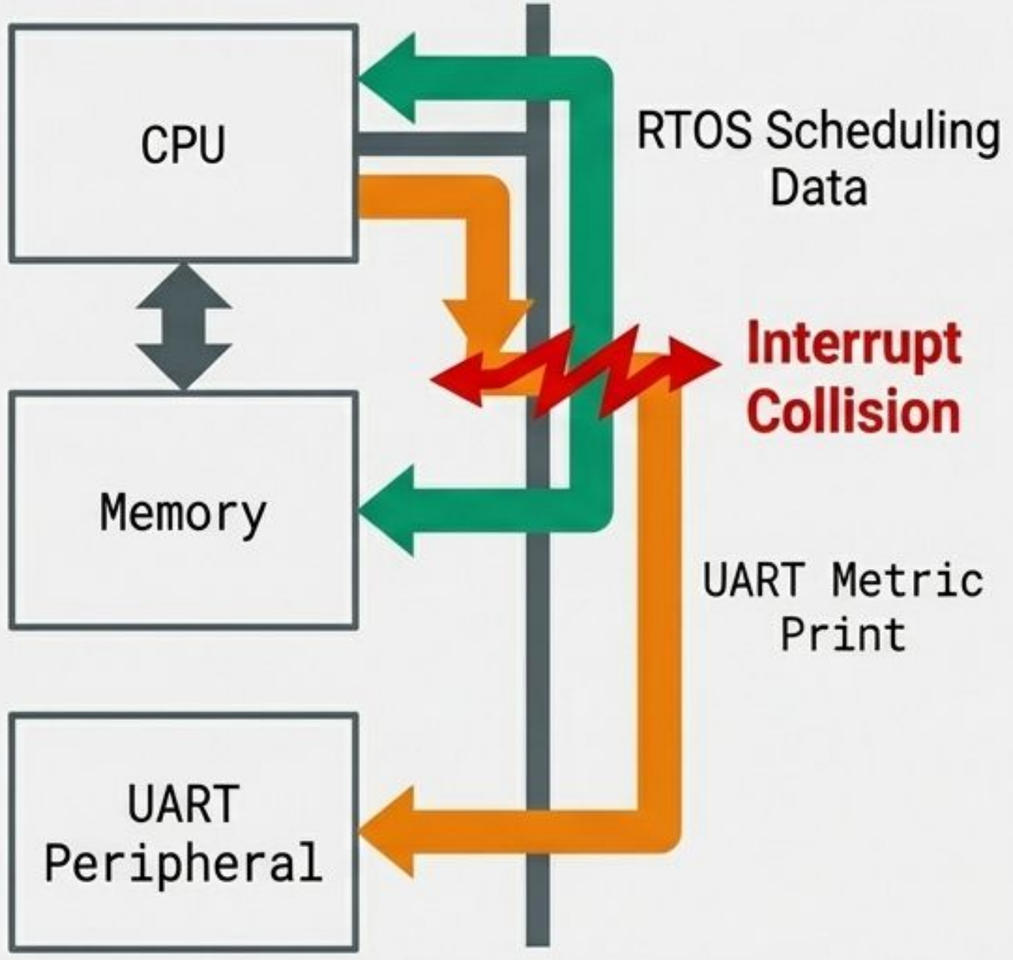
Terminal — bash — 80×24

```
[SYS] Initializing calibrated busy-spin deterministic workloads...
[SYS] ARM DWT cycle counter active. USART2 telemetry stream
initialized.

Task 1 - Prio: 3 | Jobs: 120 | Misses: 0 | DMR: 0% |
Jitter: 0.12 ms | Contention: 0 ms | Overhead: 450 cycles

Task 2 - Prio: 2 | Jobs: 85 | Misses: 0 | DMR: 0% |
Jitter: 0.45 ms | Contention: 12 ms | Overhead: 462 cycles

Task 3 - Prio: 2 | Jobs: 40 | Misses: 2 | DMR: 5% |
Jitter: 4.20 ms | Contention: 85 ms | Overhead: 510 cycles

Task %d - Prio: %lu | Jobs: %lu | Misses: %lu | DMR: %d%% |
Jitter: %lu.%02lu ms | Contention: %lu ms | Overhead: %lu cycles
```

Periodic Reporting: System state is serialized and pushed via USART2 every hyper-period, tracking Miss Ratios, precise memory contention nanoseconds, and CPU cycle overhead.

Technical Insight: The UART Contention Bottleneck

The Architecture of Contention



The Observer Effect

The Observer Effect: To analyze the scheduler, we had to print timing metrics over UART.

The Paradox: On a single-core MCU, UART interrupts stall the pipeline and compete for the exact same bus resources.

The Challenge: Measuring memory contention introduced its own unpredictable hardware contention, heavily skewing the offline data gathering. The tool used to measure interference became a source of it.

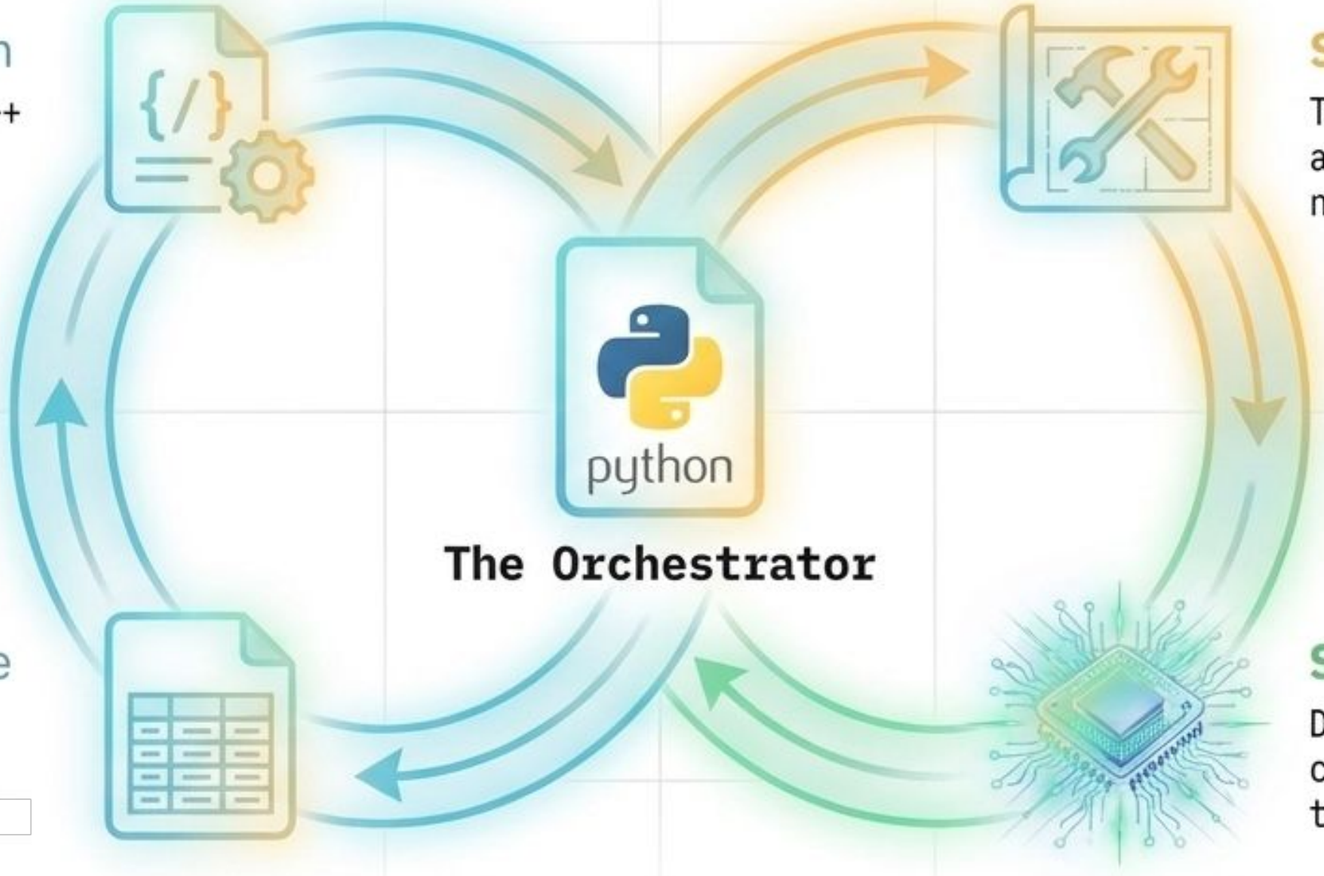
The Automated Testing Pipeline

Step 1: Code Generation

Python dynamically edits C++ configuration headers with specific test parameters.

Step 2: Build Execution

Triggers `CMakeLists.txt` to automatically compile the new firmware.



The Orchestrator

Step 3: Hardware Flash

Deploys the freshly compiled firmware directly to the STM32L452.

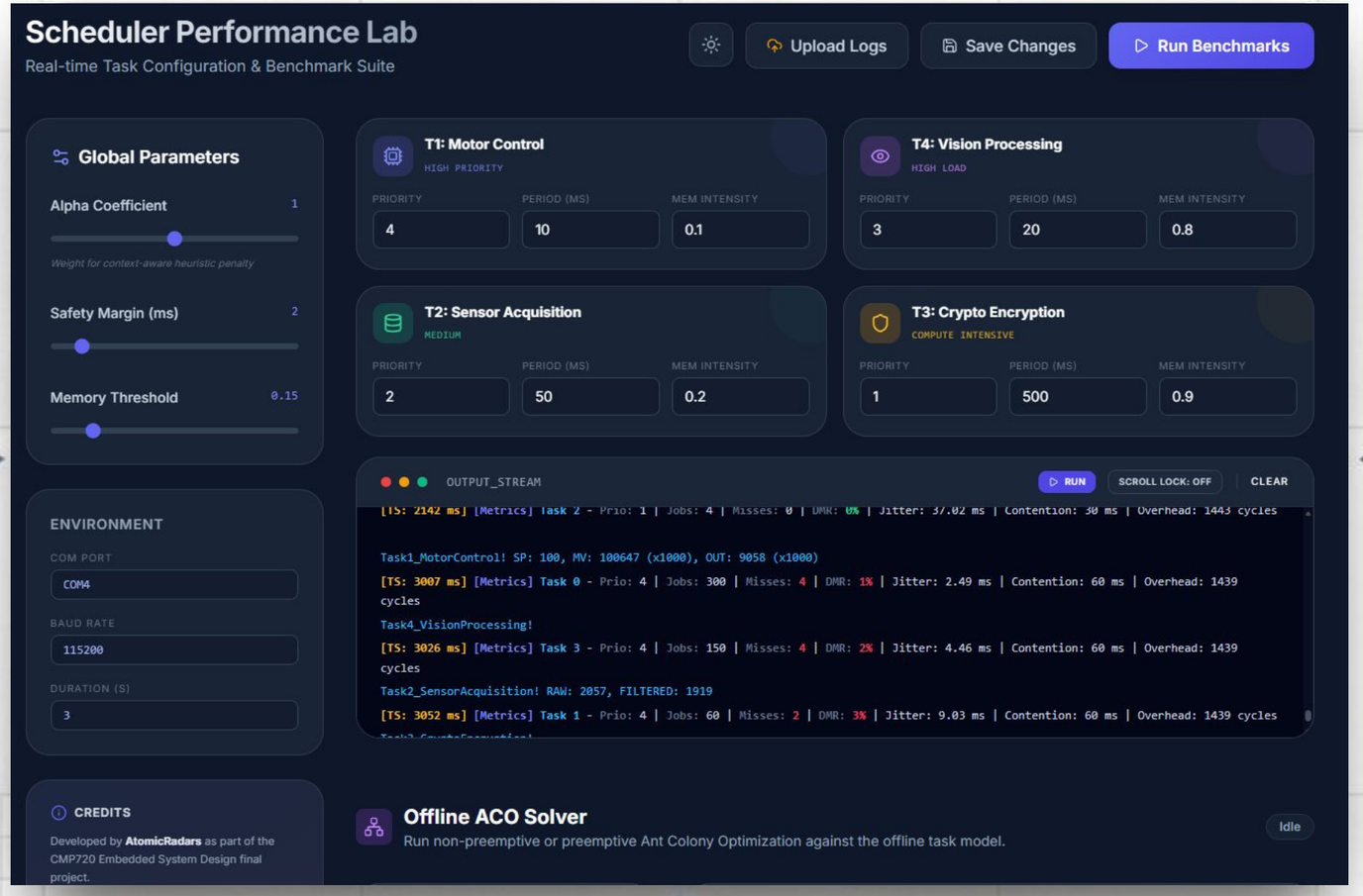
Step 4: Monitor & Parse

Automatically listens to USART2, parsing telemetry strings into clean data logs.

The orchestrator autonomously loops through all load tests across Native, EDF, and Context-Aware schedulers with zero human intervention required.

GUI Server Analytics and Final Insights

No-Code Manipulation & Theme Switch: Adjust hardware parameters without touching C++ source and toggle theme.

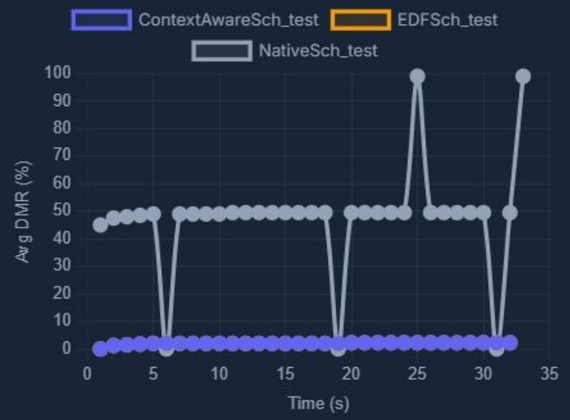


Automated Insights & Explorers: Instantly generates comparative proofs and deep-dives into metrics.

Live & Offline Execution: Fire the Python pipeline, explore parameters, and run offline ACO.

FINAL DEADLINE MISS RATIO (%)

DMR TREND OVER TIME



Upload Logs | Save Changes | Run Benchmarks

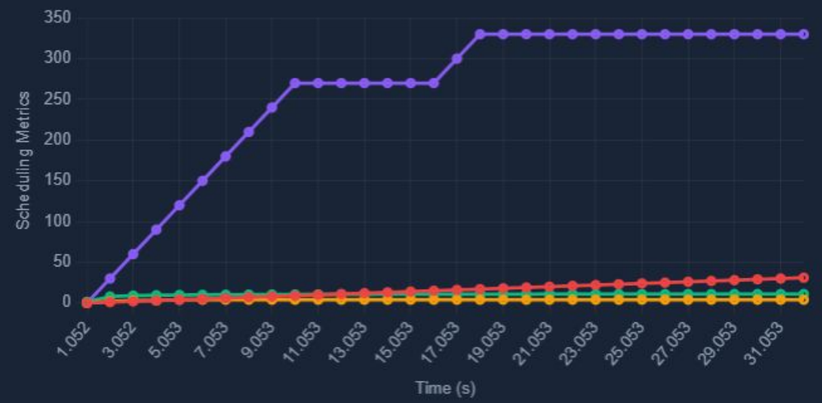
TASK EXECUTION & PARAMETER EXPLORER

Deep-dive into specific task parameters, control loop feedback, and real-time execution values.

Metrics | Parameters

Context-Aware

Sensor Acquisition (RAW / FILTERED)



Jobs Executed | Deadline Misses | Miss Ratio (DMR %)

Jitter (ms) | Contention Penalty (ms) | Decision Overhead (cyc)

EXECUTION HISTORY

Time	Jobs	DMR	RAW / FILT
1.1s	20	0%	2076 / 1899
2.1s	40	2%	1067 / 2423
3.1s	60	3%	2057 / 1919
4.1s	80	3%	1048 / 2405
5.1s	100	4%	2141 / 1926
6.1s	120	4%	1132 / 2412
7.1s	140	4%	2122 / 1949
8.1s	160	4%	1113 / 2436

Offline Benchmark

Task	Prio	Period (ms)	MemIntensity
T1 (Motor)	4	10	0.10
T2 (Sensor)	2	50	0.20
T3 (Crypto)	1	500	0.90
T4 (Vision)	3	20	0.80

Global Parameters:
Alpha: 1.00 | Safety Margin: 2 ms | Threshold: 0.15

$$P(j_a, j_b) = \alpha M_a M_b$$

Analysis of Context-Aware Scheduler

Offline Mode Active: Currently plotting historical uploaded log datasets.

Reset to Live Server

Global Parameters

Alpha Coefficient 1

Weight for context-aware heuristic penalty

Safety Margin (ms) 2

Memory Threshold 0.15

ENVIRONMENT

COM PORT: COM4

BAUD RATE: 115200

DURATION (S): 3

CREDITS

Developed by **AtomicRadars** as part of the CMP720 Embedded System Design final project.

Performance Analytics

Displaying: Loaded Log Files
Timestamp: Offline Dataset

OFFLINE RUN PARAMETERS & TASK CONFIGURATION | Run Duration: 32.6s | 19/05/2026, 18:55:06 | CLICK TO EXPAND

CRITICAL TASK JITTER (T1)

Context-Aware:	2.95 ms
EDF:	2.95 ms
Native RMS:	1.91 ms

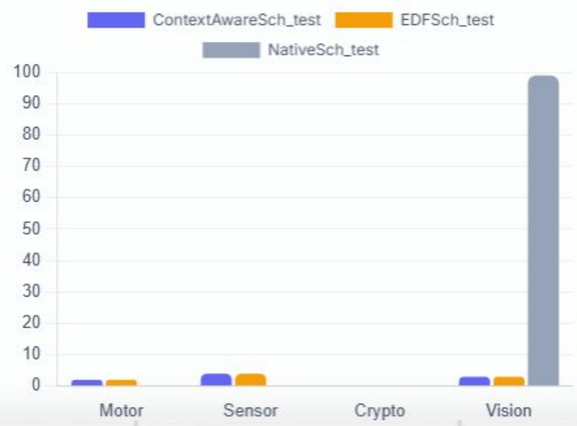
TOTAL CONTENTION PENALTY

Context-Aware:	330 ms
EDF:	720 ms
Native RMS:	0 ms

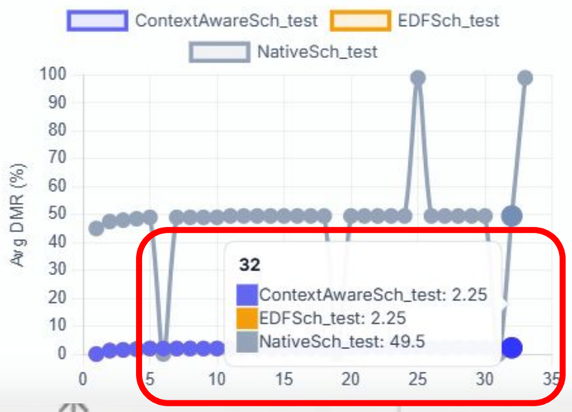
AVG DECISION OVERHEAD

Context-Aware:	1439 cyc
EDF:	1035 cyc
Native RMS:	0 cyc

FINAL DEADLINE MISS RATIO (%)



DMR TREND OVER TIME



CRITICAL TASK JITTER (T1)

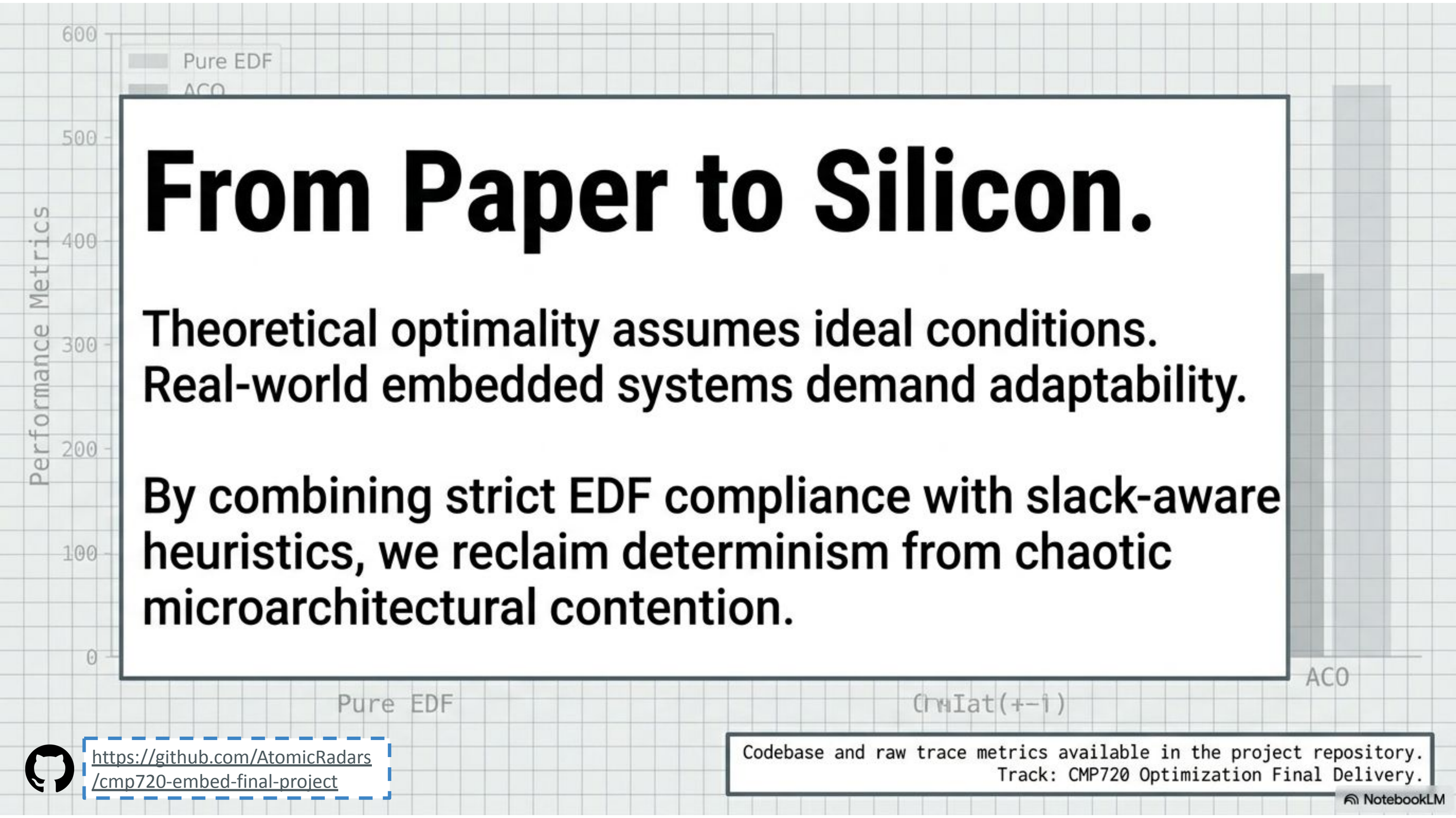
Context-Aware:	2.95 ms
EDF:	2.95 ms
Native RMS:	1.91 ms

TOTAL CONTENTION PENALTY

Context-Aware:	330 ms
EDF:	720 ms
Native RMS:	0 ms

AVG DECISION OVERHEAD

Context-Aware:	1439 cyc
EDF:	1035 cyc
Native RMS:	0 cyc



From Paper to Silicon.

Theoretical optimality assumes ideal conditions. Real-world embedded systems demand adaptability.

By combining strict EDF compliance with slack-aware heuristics, we reclaim determinism from chaotic microarchitectural contention.

https://github.com/AtomicRadars/cmp720-embed-final-project

Codebase and raw trace metrics available in the project repository.
Track: CMP720 Optimization Final Delivery.